

Indice

Introduzione.....	3
Rilevamento ping tramite Suricata.....	5
Rilevamento pacchetti Scapy.....	8
Integrazione tra Suricata e Lua.....	9
Integrazione tra Suricata e Modbus.....	13
Introduzione Modbus.....	13
Implementazione di pyModbus.....	15
Conclusioni.....	19

Introduzione

Durante questi ultimi mesi ho avuto l'opportunità di svolgere un tirocinio presso **UlisseLab**, un gruppo di ricerca nel campo della sicurezza informatica con sede nell'**Università di Bologna**.

L'argomento cardine e costante su cui è ruotata l'esperienza è [Suricata](#) e la sua integrazione con il **protocollo industriale Modbus**. Le fasi di apprendimento che ho scelto di intraprendere partono dallo studio ed acquisizione di dimestichezza con Suricata; la struttura di quest'ultimo si basa sulla creazione di regole tramite delle specifiche keywords, volte a "catturare" uno specifico tipo di traffico e riportare il log sospetto in uno specifico file dove l'utente può accedere.

L'utilizzo di Modbus è stato fondamentale in quanto funge da impalcatura perfetta per simulare un utilizzo di Suricata in ambito realistico e industriale, in quanto Modbus è formato da un architettura **client-server**. A questo punto il nostro IDS può essere collocato nel mezzo della comunicazione master-slave e rilevare possibili e sospetti attacchi al sistema.

Nel graduale processo di apprendimento sono state utilizzate delle tecnologie per comprendere al meglio il funzionamento di determinati meccanismi: inizialmente è stato utilizzato [Scapy](#), ovvero una **libreria Python** il cui scopo è quello di generare finti pacchetti grazie al supporto di vari protocolli tra cui **ARP, DNS, Dot11, TCP e Ethernet**. Inoltre nei suddetti pacchetti è stato anche possibile inserire dei payload personalizzati che, in accoppiata con delle **Suricata Keywords** finalizzate all'analisi dei payload dei pacchetti, hanno permesso un ulteriore livello di simulazione di traffico reale.

Rimanendo nell'argomento dell'analisi dei payload, Suricata offre un buon supporto al **linguaggio Lua** tramite l'inserimento nelle regole di script volti a parsificare il payload di un pacchetto. Ciò amplifica ulteriormente l'espressività che può essere raggiunta con le keywords e permette di analizzare strutture di dati più complesse.

Per creare l'infrastruttura Modbus è possibile utilizzare diversi linguaggi di programmazione e librerie annesse. In questo caso è stato utilizzato [pyModbus](#), ovvero una **libreria Python open-source** che offre **implementazioni dei protocolli Modbus** per comunicazione tra dispositivi industriali. PyModbus semplifica l'implementazione di comunicazioni Modbus tramite una serie di classi e funzioni, consentendo di creare applicazioni per monitorare e controllare dispositivi industriali in modo efficiente. Grazie alla documentazione realizzata in maniera alquanto dettagliata è stato possibile realizzare un'infrastruttura client-server stabile e semplice.

Riassumendo, durante lo svolgimento del tirocinio è stato possibile apprendere cos'è Suricata, come funziona e il modo in cui con la sua grande espressività è possibile intercettare vari tipi di traffico grazie alle numerose funzionalità che offre.

Tale espressività è stata possibile anche tramite l'apprendimento del linguaggio Lua, il quale è supportato pienamente da Suricata sia per l'analisi del payload che per il settaggio personalizzato di nuovi formati dei file di log.

Rilevamento ping tramite Suricata

[Suricata](#) si distingue nel campo della sicurezza informatica come un sistema di rilevazione e prevenzione delle intrusioni (IDS/IPS) gratuito e open source. Suricata, sviluppato e sostenuto dall'**Open Information Security Foundation (OISF)**, non solo può funzionare come un tradizionale sistema di rilevamento delle intrusioni, identificando attività sospette o malevoli all'interno di una rete, ma può anche funzionare come un sistema di prevenzione delle intrusioni, bloccando gli attacchi noti prima che possano causare danni.

Suricata può analizzare in profondità il traffico dei dati, che le consente di identificare una vasta gamma di minacce specifiche che sono collegate a diversi ambiti e protocolli. Tuttavia, il sistema di regole è una delle sue caratteristiche più notevoli perché consente la definizione di pattern di traffico o sequenze di pacchetti che potrebbero indicare un potenziale attacco o attività malevola. Suricata può generare un avviso o intraprendere un'azione specifica in base alla configurazione ogni volta che una di queste regole viene corrisposta dal traffico di rete. Queste regole possono essere derivate da set di regole predeterminati, come **Talos di Snort**, ma gli utenti possono anche scegliere di creare e applicare regole personalizzate, il che garantisce una soluzione che si adatta alle esigenze di sicurezza specifiche.

Suricata può prendere misure preventive e registrare ogni attività sospetta o malevola. Questi file di log contengono un riepilogo completo dei risultati rilevati, che include metadati come **l'indirizzo IP sorgente, l'indirizzo IP di destinazione, il tipo di attacco sospetto e altri dettagli pertinenti**. Gli analisti della sicurezza hanno bisogno di questi log per esaminare e comprendere meglio le minacce e prendere decisioni informate su come migliorare la sicurezza della rete.

Nel contesto attuale della sicurezza, Suricata è un esempio di un **IDS/IPS** di grande importanza. In un'era in cui le minacce inerenti alla sicurezza informatica sono in costante evoluzione, è fondamentale avere strumenti che consentano di identificare e reagire alle minacce in tempo reale. Per assicurarsi che le loro reti rimangano sicure e protette, le organizzazioni hanno Suricata come un potente alleato.

Per imparare e acquisire confidenza con Suricata ed il suo sistema basato sulle regole, ho iniziato con l'effettuare dei semplici **ping** dalla mia **macchina host Windows** verso una **macchina virtuale Kali Linux** e viceversa.

Il ping è uno strumento prezioso per la diagnostica di rete, che consente di valutare la connettività tra due dispositivi. Quando un computer invia un "ping" a un altro, trasmette un pacchetto di dati e attende una risposta. Questo pacchetto è chiamato **ICMP Echo**

Request. Una volta ricevuta la richiesta, l'altro computer risponde con una **ICMP Echo Response**.

Il tempo impiegato per inviare il pacchetto e ricevere la risposta viene registrato e rappresenta il **tempo di latenza** tra i due computer. Questo tempo, generalmente misurato in millisecondi, può essere utilizzato per diagnosticare problemi di rete, come il ritardo di connessione o la perdita di pacchetti.

Se non viene ricevuta alcuna risposta, significa che la connessione tra i due computer non è riuscita. Questo può indicare un problema con uno dei computer, con la rete stessa o con un dispositivo di rete intermedio, come un router o uno switch.

Dopodichè viene creata la seguente regola [ping.rules](#), collocata nel corretto percorso `/etc/suricata/rules/` ed aggiunta al file di configurazione **suricata.yaml**:

alert icmp any any -> any any (msg:"PING RILEVATO"; sid:101; rev:1)

- **alert**: alla rilevazione del pacchetto indicato, Suricata scriverà il log sospetto nei file di competenza
- **icmp**: è il protocollo utilizzato dal ping
- **any any -> any any**: indica che viene controllato il traffico da una qualsiasi porta di un qualsiasi indirizzo ad una porta qualsiasi di un indirizzo qualsiasi.
- **msg**: messaggio personalizzato
- **sid**: id univoco della regola
- **rev**: versione della regola

Inserendo "**alert**" come keyword, implicitamente Suricata viene impostato come **IDS (Intrusion Detection System)**, perciò una volta che il ping viene rilevato, il sistema non lo blocca ma si occupa soltanto di avvertire l'utente del log sospetto. Altre azioni possibili sono "**drop**", che blocca e scarta silenziosamente il pacchetto, "**pass**", che permette al pacchetto di continuare senza ulteriori ispezioni e "**reject**", che interrompe il traffico e lo notifica al mittente.

Una volta inviato il ping con il comando da shell (**ping IP**), se è andato tutto a buon fine e Suricata è stato settato nel modo corretto, mi aspetto che nei file **fast.log** e **eve.json** situati nella cartella `/var/log/suricata/` compaiano i log inerenti a quei determinati ping.

In una nuova finestra verrà inserito **sudo tail -f /var/log/suricata/fast.log** che, grazie al comando **tail -f**, permette di tenere d'occhio live il file in questione.

Una volta lanciato il ping dalla macchina Kali, nel file fast.log saranno comparsi **due log**, in quanto il ping prevede un messaggio di andata e di ritorno.

```
(kali@kali)-[~]
$ sudo tail -f /var/log/suricata/fast.log
08/06/2023-11:15:06.189531  [**] [1:101:1] PING RILEVATO [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.1:0
08/06/2023-11:15:06.190086  [**] [1:101:1] PING RILEVATO [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.1:0 → 192.168.56.103:0
```

Log rilevato nel file fast.log

Come previsto i file contengono ciò che mi aspettavo, cioè: la **data di rilevamento dell>alert**, l'**id** e la **versione della regola**, un **messaggio personalizzabile**, la **classificazione della regola** ed il suo **livello di priorità**, **protocollo rilevato** ed **indirizzi e porte dei dispositivi di origine e destinazione**.

E' possibile analizzare i log anche nel file **eve.json**, ma in primis è necessario installare **jq**. Quest'ultimo è un processore leggero e flessibile di linea di comando per JSON che fornisce la capacità di filtrare, trasformare, manipolare e creare dati JSON.

JSON (JavaScript Object Notation) è un formato leggero per lo scambio di dati. È facile da leggere e scrivere per gli esseri umani ed è comunemente usato per trasferire dati tra un server e un'applicazione web. È un formato di testo completamente indipendente dal linguaggio, ma utilizza convenzioni familiari ai programmatori di C, C++, Java, JavaScript, Python e molte altre famiglie di linguaggi. Si basa su due costrutti: un insieme di coppie **nome/valore** e un **elenco ordinato di valori**. Senza di esso aprendo il file eve.json con un semplice cat o un tail verranno restituite un insieme di stringhe senza formattazione e piuttosto scomodo da leggere.

```
{ "timestamp": "2023-08-07T19:16:14.736029+0000", "event_type": "stats", "stats": { "uptime": 640, "capture": { "kernel_packet_s": 2184, "kernel_drops": 0, "errors": 0 }, "decoder": { "pkts": 2185, "bytes": 289336, "invalid": 0, "ipv4": 2164, "ipv6": 17, "ethernet": 2185, "chdcl": 0, "raw": 0, "null": 0, "ssl": 0, "tcp": 2072, "udp": 92, "sctp": 0, "icmpv4": 0, "icmpv6": 7, "ppp": 0, "pppoe": 0, "geneve": 0, "gre": 0, "vlan": 0, "vlan_qinq": 0, "vxlan": 0, "ieee8021ah": 0, "teredo": 0, "ipv4_in_ipv6": 0, "ipv6_in_ipv6": 0, "mpls": 0, "avg_pkt_size": 132, "max_pkt_size": 1514, "max_mac_addr_src": 0, "max_mac_addr_dst": 0, "erspan": 0, "event": { "ip4": { "pkt_too_small": 0, "hlen_too_small": 0, "iplen_smaller_than_hlen": 0, "trunc_pkt": 0, "opt_invalid": 0, "opt_invalid_len": 0, "opt_malformed": 0, "opt_bad_required": 0, "opt_eol_required": 0, "opt_duplicate": 0, "opt_unknown": 0, "wrong_ip_version": 0, "icmpv6": 0, "frag_pkt_too_large": 0, "frag_overlap": 0, "frag_ignored": 0, "icmpv4": { "pkt_too_small": 0, "unknown_type": 0, "unknown_code": 0, "ipv4_trunc_pkt": 0, "ipv4_unknown_ver": 0, "icmpv6": { "unknown_type": 0, "unknown_code": 0, "pkt_too_small": 0, "ipv6_unknown_version": 0, "ipv6_trunc_pkt": 0, "mld_message_with_invalid_hl": 0, "unassigned_type": 0, "experimental_option_type": 0, "ipv6": { "pkt_too_small": 0, "trunc_pkt": 0, "trunc_exthdr": 0, "exthdr_dupl_fh": 0, "exthdr_useless_fh": 0, "exthdr_r_dupl_rh": 0, "exthdr_dupl_hh": 0, "exthdr_dupl_dh": 0, "exthdr_dupl_ah": 0, "exthdr_dupl_eh": 0, "exthdr_invalid_optlen": 0, "wrong_ip_version": 0, "exthdr_ah_res_not_null": 0, "hopopts_unknown_opt": 0, "hopopts_only_padding": 0, "dstopts_unknown_opt": 0, "dstopts_only_padding": 0, "rh_type": 0, "zero_len_padn": 0, "fh_non_zero_reserved_field": 0, "data_after_none_header": 0, "unknown_next_header": 0, "icmpv4": 0, "frag_pkt_too_large": 0, "frag_overlap": 0, "frag_ignored": 0, "ipv4_in_ipv6_wrong_version": 0, "ipv6_in_ipv6_too_small": 0, "ipv6_in_ipv6_wrong_version": 0, "tcp": { "pkt_too_small": 0, "hlen_too_small": 0, "invalid_optlen": 0, "opt_invalid_len": 0, "opt_duplicate": 0, "udp": { "pkt_too_small": 0, "hlen_too_small": 0, "hlen_invalid": 0, "ssl": { "pkt_too_small": 0, "ethernet": { "pkt_too_small": 0, "ppp": { "pkt_too_small": 0, "vju_pkt_too_small": 0, "ip4_pkt_too_small": 0, "ip6_pkt_too_small": 0, "wrong_type": 0, "unsup_proto": 0, "pppoe": { "pkt_too_small": 0, "wrong_code": 0, "malformed_tags": 0, "gre": { "pkt_too_small": 0, "wrong_version": 0, "version0_recur": 0, "version0_no_flags": 0, "version0_hdr_too_big": 0, "version0_malformed_sre_hdr": 0, "version1_checksum": 0, "version1_route": 0, "version1_ssr": 0, "version1_recur": 0, "version1_flags": 0, "version1_no_key": 0, "version1_wrong_protocol": 0, "version1_malformed_sre_hdr": 0, "version1_hdr_too_big": 0, "vlan": { "header_too_small": 0, "unknown_type": 0, "too_many_layers": 0, "ieee8021ah": { "header_too_small": 0, "ipraw": { "invalid_ip_version": 0, "ltnull": { "pkt_too_small": 0, "unsupported_type": 0, "sctp": { "pkt_too_small": 0, "bad_label": 0, "bad_label_router_alert": 0, "bad_label_implicit_nul": 0, "bad_label_reserved": 0, "unknown_payload_type": 0, "vxlan": { "unknown_payload_type": 0, "geneve": { "unknown_payload_type": 0, "erspan": { "header_too_small": 0, "unsupported_version": 0, "too_many_vlan_layers": 0, "dce": { "pkt_too_small": 0, "flow": { "memcap": 0, "tcp": 3, "udp": 47, "icmpv4": 0, "icmpv6": 5, "tcp_reuse": 0, "get_used": 0, "get_used_eval": 0, "get_used_eval_reject": 0, "get_used_eval_busy": 0, "get_used_failed": 0, "wrk": { "spare_sync_avg": 100, "spare_sync": 2, "spare_sync_incomplete": 0, "spare_sync_empty": 0, "flows_evicted_needs_work": 1, "flows_evicted_pkt_inject": 2, "flows_evicted": 2, "flows_injected": 1, "mgr": { "full_hash_pass": 3, "closed_pruned": 0, "new_pruned": 0, "est_pruned": 0, "bypassed_pruned": 0, "ro_ws_maxlen": 1, "flows_checked": 71, "flows_timeout": 41, "flows_timeout_inuse": 30, "flows_timeout_inuse": 0, "flows_evicted_needs_work": 1, "spare": 9829, "emerg_mode_entered": 0, "emerg_mode_over": 0, "memuse": 7474304, "defrag": { "ip4": { "fragments": 0, "reassembled": 0, "timeouts": 0, "ip6": { "fragments": 0, "reassembled": 0, "timeouts": 0, "max_frag_hits": 0, "flow_bypassed": { "local_pkts": 0, "local_bytes": 0, "local_capture_pkts": 0, "local_capture_bytes": 0, "closed": 0, "pkts": 0, "bytes": 0, "tcp": { "sessions": 3, "ssn_memcap_drop": 0, "pseudo": 0, "pseudo_failed": 0, "invalid_checksum": 0, "no_flow": 0, "syn": 3, "synack": 3, "rst": 1, "midstream_pickups": 0, "pkt_on_wrong_thread": 0, "segment_memcap_drop": 0, "stream_depth_reached": 0, "reassembly_gap": 0, "overlap": 0, "overlap_diff_data": 0, "insert_data_normal_fail": 0, "insert_data_overlap_fail": 0, "insert_list_fail": 0, "memuse": 1146880, "reassembly_memuse": 196608, "detect": { "engines": { "id": 0, "last_reload": "2023-08-07T19:05:34.566863+0000", "rules_loaded": 5, "rules_failed": 0 }, "alert": 0, "app_layer": { "flow": { "http": 0, "ftp": 0, "smtp": 0, "tls": 0, "ssh": 3, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "modbus": 0, "nfs_tcp": 0, "ntp": 46, "ftp_data": 0, "ftp": 0, "ikev2": 0, "krb5_tcp": 0, "dhcpc": 0, "smp": 0, "sip": 0, "rfb": 0, "mqt": 0, "met": 0, "rdp": 0, "failed_tcp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0, "failed_udp": 1, "ttx": { "http": 0, "ftp": 0, "smtp": 0, "ssh": 0, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "modbus": 0, "nfs_tcp": 0, "ntp": 46, "ftp_data": 0, "ftp": 0, "ikev2": 0, "krb5_tcp": 0, "dhcpc": 0, "smp": 0, "sip": 0, "rfb": 0, "mqt": 0, "rdp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0, "expectations": 0 }, "http": { "memuse": 0, "memcap": 0 }, "ftp": { "memuse": 0, "memcap": 0 } } } } } } } } } } } } } } } }
```

L'installazione avviene con il comando ***sudo apt install jq***. Tramite le funzionalità del processore è possibile selezionare solo le chiavi JSON che vogliamo controllare, in questo caso grazie alla keyword **select** è possibile selezionare solo l'**event_type** che ci interessa, in questo caso trattandosi di un **alert** inseriremo **jq 'select(.event_type=="alert")'**.

```
(kali@kali)-[~]
$ sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
[sudo] password for kali:
{
  "timestamp": "2023-08-06T11:50:30.496500-0400",
  "flow_id": 1850976301327939,
  "in_iface": "eth0",
  "event_type": "alert",
  "src_ip": "192.168.56.103",
  "src_port": 0,
  "dest_ip": "192.168.56.1",
  "dest_port": 0,
  "proto": "ICMP",
  "icmp_type": 8,
  "icmp_code": 0,
  "pkt_src": "wire/pcap",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 101,
    "rev": 1,
    "signature": "PING RILEVATO",
    "category": "",
    "severity": 3
  },
  "direction": "to_server",
  "flow": {
    "pkts_toserver": 1,
    "pkts_toclient": 0,
    "bytes_toserver": 98,
    "bytes_toclient": 0,
    "start": "2023-08-06T11:50:30.496500-0400",
    "src_ip": "192.168.56.103",
    "dest_ip": "192.168.56.1"
  }
}
```

Log del ping dalla macchina Linux all'host Windows

Si nota inoltre come ora le informazioni siano decisamente più comprensibili, ciò aiuta di gran lunga ad analizzare il log per trovare possibili anomalie.

Rilevamento pacchetti Scapy

Dopo questo primo approccio con i ping, ho deciso l'apprendimento di Suricata è proseguito con l'uso di Scapy.

Scapy è una **libreria Python** utilizzata per la manipolazione dei pacchetti di rete. È un potente strumento per la **costruzione**, l'**invio**, la **cattura** e l'**analisi di pacchetti di rete**. Offre inoltre la possibilità di decodificare o codificare a livello di protocollo una vasta gamma di protocolli di rete.

Scapy viene utilizzato per una serie di applicazioni legate alla sicurezza e alla diagnostica della rete. Può aiutare a identificare problemi di rete, testare la sicurezza della rete, analizzare il traffico e comprendere come funzionano i protocolli di rete.

Con Scapy, è possibile creare pacchetti di rete personalizzati da zero, consentendo un controllo dettagliato su ogni aspetto del pacchetto. Questa funzione è particolarmente utile per testare la sicurezza di una rete, poiché permette di simulare varie tipologie di attacchi e di testare la resistenza della rete a tali attacchi.

Scapy supporta una vasta gamma di protocolli, tra cui **TCP, UDP, ICMP, ARP e Ethernet**, solo per citarne alcuni. Ciò significa che può essere utilizzato per lavorare con praticamente qualsiasi tipo di pacchetto di rete.

Oltre alla creazione e manipolazione dei pacchetti, Scapy offre anche funzioni per catturare pacchetti di rete, effettuare tracciamenti di rotta e eseguire scansioni delle porte. Queste funzioni, combinate con la sua capacità di manipolare pacchetti, rendono Scapy uno strumento molto versatile per la gestione e l'analisi delle reti.

Infine, Scapy può essere utilizzato come una libreria stand-alone in uno script Python, o come un programma interattivo da un terminale. Questo fatto lo rende estremamente flessibile e adattabile a una serie di diverse situazioni e applicazioni.

In questo specifico caso Scapy è stato utilizzato per creare dei pacchetti con un payload personalizzato che contiene delle finte informazioni riguardo macchinari industriali, tutto ciò grazie allo script [tcp_test.py](#).


```

addr = "192.168.1.52"
iface = "wlan0"
payload = "--Info--\nCPU Usage:100%; \nTemperature: 90°; \nHumidity: 50%;"

print("sending on interface %s to %s" % (iface, str(addr)))

pkt = Ether(src=get_if_hwaddr(iface), dst='ff:ff:ff:ff:ff:ff')
pkt = IP(dst="192.168.56.101")/TCP(dport=8082, sport=random.randint(49152,65535))/payload
pkt.show2()
send(pkt)

```

Codice script Scapy

L'obiettivo iniziale è quello di leggere i valori numerici dopo le stringhe e controllare che non superino certi valori critici che potrebbero significare un possibile attacco o un'anomalia casuale. In questo caso la regola genererà un apposito log nei file citati in precedenza. Tramite la Suricata Keyword **content** è possibile analizzare il payload di un pacchetto, ma in questo caso tale metodo risulta molto rigido e macchinoso e soprattutto non offre alcun tipo di flessibilità ed espressibilità nella manipolazione del payload.

Per raggiungere tale obiettivo viene sfruttato un linguaggio di programmazione per cui Suricata offre un supporto piuttosto completo, ovvero Lua.

Integrazione tra Suricata e Lua

[Lua](#) è un linguaggio di programmazione che è stato creato nel 1993 presso l'**Universidade Federal do Rio de Janeiro** in Brasile. È nato dal gruppo di ricerca Tecgraf, il Gruppo di Tecnologia in Computer Graphics, guidato da Roberto Ierusalimsky, Luiz Henrique de Figueiredo e Waldemar Celes.

Lua è un linguaggio di scripting, progettato principalmente per l'estensione di software e per l'incorporamento di scripting in applicazioni. È **open-source** e il suo design leggero, insieme alla sua capacità di essere incorporato, lo rende particolarmente popolare nel campo dello sviluppo di videogiochi e nelle applicazioni di rete.

Il linguaggio Lua è noto per la sua semplicità. Ha una sintassi pulita e semplice, che rende facile l'apprendimento per i nuovi programmatori. Tuttavia, nonostante ciò, è anche molto potente e flessibile. Lua supporta sia la programmazione procedurale che quella orientata agli oggetti, e include funzionalità come il controllo del flusso, le strutture dati e le funzioni di ordine superiore.

Un'altra caratteristica importante è la sua portabilità. È scritto in **C ANSI**, il che significa che può essere eseguito su quasi tutte le piattaforme e i sistemi operativi. Inoltre, Lua è progettato per essere molto efficiente, con una piccola memoria footprint e un motore di runtime veloce.

Lua include anche un potente sistema per la gestione delle tabelle, che possono funzionare come array, dizionari, set e altre strutture dati. Questo sistema di tabelle, insieme al supporto di Lua per le funzioni di prima classe, rende possibile utilizzare tecniche di programmazione funzionale avanzate.

Infine, uno dei principali vantaggi di Lua è la sua comunità di sviluppatori. Ci sono molte librerie e framework disponibili per Lua, che estendono le sue funzionalità in molte aree diverse, dai database ai videogiochi ai servizi web.

Grazie a Lua è stato possibile leggere il payload del pacchetto inviato e contemporaneamente, grazie all' integrazione che Suricata offre verso tale linguaggio di programmazione, permette di far "scattare" una regola grazie alla keyword **lua:script.lua** da settare nell'apposita regola. Se i valori della CPU, temperatura e umidità superano una certa soglia contemporaneamente, lo script Lua restituisce il valore 1 e attiva la regola Suricata.

Per far sì che Suricata rilevi il pacchetto vado ad aggiungere la seguente regola suricata [test_scapy.rules](#):

alert tcp any any -> any any (msg:" Too much CPU usage/high temperature/high humidity. Possible attack!!!"; luajit: lua_script.lua; sid:102; rev:1)

- **tcp** -> in quando il pacchetto creato con Scapy simula il payload di un pacchetto tcp.
- **luajit: lua_script.lua** -> script lua

Da notare come la keyword usata non sia solo **lua** ma **luajit**. Per capire il motivo dietro ciò, si va prima a lanciare da shell il comando **suricata -build-info**.

```
Prelude support:      no
PCRE jit:             yes
LUA support:          yes, through luajit
liblua:               yes
GeoIP2 support:       yes
```

Output suricata -build-info

Quest' ultimo elenca le tecnologie supportate sul proprio dispositivo da Suricata. Nella riga dedicata a Lua viene inserito: **yes, through Luajit**. Luajit è una versione più veloce e ottimizzata di Lua ed inizialmente Suricata supportava solo Luajit, in seguito è stato

aggiunto anche il supporto per Lua. Per sapere a quale libreria, Lua o Luajit, è collegato Suricata sul proprio dispositivo è possibile lanciare il comando: **ldd** **<percorso/del/binario/suricata>**.

```
(edomari@edomari)~$ ldd /usr/bin/suricata | grep lua
liblua_jit-5.1.so.2 => /lib/x86_64-linux-gnu/liblua_jit-5.1.so.2 (0x00007fac0f695000)
```

Esempio dell'output del comando ldd sulla macchina virtuale Kali

Nota: lanciando da solo il comando *ldd* vengono elencate le librerie condivise che ogni programma utilizza, viene aggiunto alla pipeline anche il comando **grep** per selezionare solo la libreria inerente a Lua.

Lo script Lua dovrà essere posizionato nella cartella */etc/suricata/rules* per far sì che il sistema lo trovi al momento della chiamata.

L'integrazione tra Suricata e Lua per lo scripting segue un formato preciso; una **funzione init**, che serve per inizializzare lo script e specificare il buffer che necessita ispezione, e la **funzione match** che si occupa di restituire alla regola Suricata il valore 1 se le condizioni verificate corrispondono, 0 altrimenti.

- packet – entire packet, including headers
- payload – packet payload (not stream)
- buffer – the current sticky buffer
- http.uri
- http.uri.raw
- http.request_line
- http.request_headers
- http.request_headers.raw
- http.request_cookie
- http.request_user_agent
- http.request_body
- http.response_headers
- http.response_headers.raw
- http.response_body
- http.response_cookie

Buffer attualmente supportati da Suricata
alla versione 7.0.0

A questo punto ho deciso di aggiungere un livello di specificità in più alla regola precedentemente scritta. Nella prossima regola controllerò sempre che i valori della CPU, temperatura e umidità non superino la precedente soglia, ma in più aggiungo alla regola [test_scapy 2.rules](#) che per farla scattare la soglia non deve essere superata per più di due volte in 3 minuti.

alert tcp any any -> any any (msg:"Too much CPU usage/high temperature/high humidity. Possible attack!!!"; luajit:prova1.lua; threshold: type threshold, track by_src, count 2, seconds 180; sid:103; rev:1;)

- ***threshold: type threshold, track by_src*** -> imposta che il limite sia una soglia
- ***count 2*** -> indica il numero di volte che la regola deve verificarsi per scattare
- ***seconds 180*** -> fa scattare la regola se si verifica entro la soglia di secondi (180)

Lanciamo quindi il comando ***sudo python3 tcp_test.py*** per 4 volte e andiamo a verificare tramite il comando ***sudo tail -f /var/log/suricata/fast.log*** se il risultato è quello previsto.

```
08/28/2023-04:59:46.570664  [**] [1:103:1] Too much CPU usage/high temperature/high humidity. Possible attack!!! [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.15:41496 → 192.168.1.201:502
08/28/2023-04:59:46.571495  [**] [1:103:1] Too much CPU usage/high temperature/high humidity. Possible attack!!! [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.1.201:502 → 10.0.2.15:41496
```

Test rilevamento pacchetti Scapy

Come atteso, nonostante i valori inerenti alla temperatura, uso CPU e umidità siano troppo elevati, la regola genera due log solo se si verifica più di due volte in 2 minuti.

Integrazione tra Suricata e Modbus

Dopo aver acquisito confidenza con Suricata tramite la rilevazione dei ping prima e in seguito l'integrazione con Scapy e Lua, è arrivato il momento di concentrarsi sul **protocollo Modbus** e il suo **utilizzo in accoppiata con Suricata**. Prima di continuare è necessario spiegare cos'è Modbus e le sue caratteristiche principali.

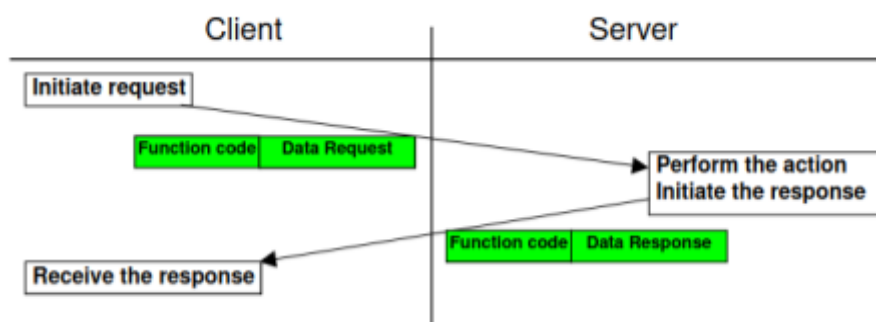
Introduzione Modbus

I protocolli di comunicazione sono essenziali nell'automazione industriale in quanto consentono ai dispositivi e ai sistemi di scambiare dati ed eseguire azioni coordinate. Questi protocolli definiscono regole e standard per la trasmissione, la ricezione e l'interpretazione dei dati, assicurando una comunicazione senza interruzioni tra dispositivi di produttori diversi. Stabiliscono un linguaggio e un formato comune per la

comunicazione, facilitando l'interoperabilità e l'integrazione in ambienti industriali complessi.

Il protocollo **Modbus**, sviluppato da **Modicon** (ora Schneider Electric) nel 1979, è stato uno dei primi protocolli di comunicazione ampiamente adottati nel settore dell'automazione industriale. Inizialmente progettato per la comunicazione seriale su connessioni **RS-232**, Modbus si è evoluto nel corso degli anni per supportare diverse modalità di comunicazione, tra cui **RS-485**, **Ethernet** e **TCP/IP**. La natura aperta e la semplicità del protocollo hanno contribuito alla sua adozione diffusa e alla sua compatibilità con un'ampia gamma di dispositivi e sistemi.

Modbus segue un'**architettura client-server**, in cui un dispositivo **Modbus master** avvia la comunicazione con uno o più dispositivi **Modbus slave**. Il master Modbus invia comandi o richieste agli slave, che rispondono con i dati richiesti o eseguono le azioni specificate. Questa architettura consente uno scambio di dati efficiente e organizzato tra i dispositivi, permettendo il controllo e il monitoraggio in tempo reale nelle applicazioni industriali.



Comunicazione client/server in Modbus

Il modello di dati Modbus definisce come i dati sono organizzati e rappresentati all'interno del protocollo. Modbus supporta diversi tipi di dati, tra cui **input registers, coils, holding registers e discrete inputs**. I discrete inputs e i coils rappresentano **dati binari**, mentre gli input registers e gli holding registers memorizzano **valori analogici o numerici**. Ogni tipo di dati ha uno schema di indirizzamento specifico, che consente ai dispositivi di accedere e manipolare i dati utilizzando i codici funzione Modbus.

I codici funzione Modbus sono comandi standardizzati utilizzati per eseguire operazioni specifiche all'interno del protocollo. Questi codici funzione consentono ai dispositivi Modbus di leggere o scrivere dati, controllare il comportamento del dispositivo ed eseguire attività diagnostiche. Alcuni codici funzione comunemente utilizzati sono **Read Coils (0x01)**, **Read Input Registers (0x04)**, **Write Single Coil (0x05)** e **Write Multiple**

Registers (0x10). Ogni codice funzione ha uno scopo specifico ed è supportato dai dispositivi Modbus per una comunicazione efficace.

Il protocollo Modbus supporta diverse modalità di comunicazione per soddisfare i vari requisiti dell'automazione industriale. Le modalità più comuni sono **Modbus RTU** e **Modbus ASCII**, che utilizzano la comunicazione seriale su connessioni **RS-232** o **RS-485**. Il Modbus RTU è noto per la sua efficienza e velocità, mentre il Modbus ASCII garantisce l'integrità dei dati tramite checksum. Con l'avvento di **Ethernet e TCP/IP**, il Modbus TCP ha guadagnato popolarità come modalità di comunicazione affidabile e ad alta velocità, consentendo una perfetta integrazione con le moderne reti industriali.

Il protocollo Modbus offre diversi vantaggi nelle applicazioni di automazione industriale. La sua semplicità e apertura facilitano l'integrazione e la compatibilità con un'ampia gamma di dispositivi, indipendentemente dal produttore. Modbus supporta anche un efficiente scambio di dati, consentendo il controllo e il monitoraggio in tempo reale. Inoltre, la versatilità e la scalabilità del protocollo lo rendono adatto sia ai sistemi su piccola scala che a quelli su larga scala.

Implementazione di pyModbus

PyModbus è una **libreria Python** che fornisce un'implementazione del protocollo Modbus. Offre un modo comodo ed efficiente per sviluppare applicazioni di comunicazione Modbus utilizzando il linguaggio di programmazione Python. pyModbus semplifica il processo di integrazione delle funzionalità Modbus nei sistemi di automazione industriale basati su Python, consentendo agli sviluppatori di leggere e scrivere facilmente i dati dai dispositivi Modbus.

pyModbus offre una serie di caratteristiche che lo rendono una scelta popolare per l'implementazione di Modbus in Python. Alcune caratteristiche chiave come ad esempio il supporto di diverse varianti di Modbus, tra cui **Modbus TCP**, **Modbus RTU** e **Modbus ASCII**. Questa versatilità consente agli sviluppatori di scegliere la modalità di comunicazione appropriata in base ai loro requisiti specifici.

PyModbus offre funzionalità sia di master che di slave Modbus. Gli sviluppatori possono utilizzarlo per implementare dispositivi Modbus **master** che avviano la comunicazione con Modbus **slave** e dispositivi Modbus slave che rispondono alle richieste dei Modbus master.

Inoltre supporta sia la modalità di comunicazione sincrona che quella asincrona. In modalità sincrona, l'esecuzione del codice è bloccata fino alla ricezione di una risposta dal dispositivo Modbus. La modalità asincrona, invece, permette una comunicazione non bloccata, consentendo agli sviluppatori di eseguire altre attività in attesa di una risposta.

PyModbus supporta un'ampia gamma di codici funzione Modbus, consentendo agli sviluppatori di leggere e scrivere vari tipi di dati dai dispositivi Modbus. Alcuni codici funzione comunemente utilizzati sono **Read Coils**, **Read Input Registers**, **Write Single Coil** e **Write Multiple Registers**.

Per utilizzare pyModbus si possono seguire i seguenti step. Il primo passo consiste nell'installare la libreria pyModbus. Questo può essere fatto utilizzando pip, il programma di installazione dei pacchetti Python, a sua volta installabile eseguendo il comando `"sudo apt install python3-pip"`, eseguendo il comando `"pip install -U pyModbus"` nella riga di comando.

Una volta installata, la libreria può essere importata nello script Python utilizzando l'istruzione **"import"**. In questo modo, tutte le funzioni e le classi necessarie sono disponibili per l'uso nello script.

Per stabilire una connessione con un dispositivo Modbus, lo script deve creare un oggetto client Modbus. Questo può essere fatto istanziando un oggetto **ModbusTcpClient** per la **comunicazione Modbus TCP** o un oggetto **ModbusSerialClient** per la comunicazione Modbus RTU o Modbus ASCII.

Dopo aver stabilito la connessione, lo script può inviare richieste Modbus per leggere o scrivere dati dal dispositivo Modbus. Per eseguire l'operazione desiderata, è possibile utilizzare il codice funzione appropriato insieme ai parametri richiesti.

Una volta inviata una richiesta, lo script attende la risposta del dispositivo Modbus. La risposta può essere recuperata utilizzando il metodo appropriato fornito dalla libreria pyModbus. Lo script può quindi estrarre i dati richiesti dalla risposta ed eseguire ulteriori elaborazioni o analisi.

Una volta eseguite tutte le operazioni richieste, è importante chiudere la connessione Modbus per liberare le risorse del sistema. Per farlo, è necessario richiamare il metodo **"close()"** sull'oggetto client Modbus.

Per creare l'infrastruttura client-server di Modbus è stato molto utile il consulto della [documentazione pyModbus](#), la quale oltre a dare una descrizione dettagliata di ogni funzione e metodo volto a leggere/scrivere sui registri, fornisce una buona quantità di esempi di client e server già creati e pronti per essere utilizzati.

In questo caso, sono stati utilizzati come base gli esempi inerenti al Modbus payload Server e Modbus payload Client, in quanto il focus è quello che il client e server si scambino dei messaggi tramite stringhe come ad esempio Temperature: 68°, Humidity: 76% ecc. Per rendere più realistica la comunicazione le stringhe citate prima verranno mescolate con delle stringhe casuali; grazie allo script Lua infatti sarà possibile analizzare

solo dei determinati pattern. Il payload che verrà inserito ciclicamente e casualmente nel messaggio Modbus sarà scritto nei registri. Questi ultimi sono delle posizioni di memoria specifici in un dispositivo Modbus che possono essere letti o scritti da un altro dispositivo. Ci sono diversi tipi di registri Modbus, ognuno con uno scopo specifico:

- **Registri di Input (Input Registers):** Questi registri, anche conosciuti come registri di lettura, sono utilizzati per memorizzare i dati provenienti da dispositivi di input come sensori. Gli input registers possono essere solo letti, non scritti, da un dispositivo client.
- **Registri Holding (Holding Registers):** Possono essere sia letti che scritti da un dispositivo client. Essi sono usati per contenere informazioni o dati che possono essere utilizzati o modificati da un altro dispositivo. Ad esempio, un dispositivo potrebbe scrivere un valore in un holding register per impostare la velocità di un motore o la temperatura di un riscaldatore.
- **Coils:** I coils sono registri a 1 bit che possono essere letti o scritti da un dispositivo client. Questi sono tipicamente usati per controllare i dispositivi di output, come relè. Per esempio, scrivere un "1" in un coil potrebbe attivare un relè, mentre scrivere un "0" lo spegnerebbe.
- **Input Discreti (Discrete Inputs):** Questi registri, simili ai coils, sono a 1 bit e possono essere solo letti, non scritti, da un dispositivo client. Sono usati per memorizzare lo stato di dispositivi di input discreti, come interruttori o pulsanti.

In questo caso il payload verrà scritto negli holding registers e sarà composto da un insieme di stringhe. Per generare il payload fittizio è stata utilizzata questa funzione in Python che prende in input le stringhe che si vuole analizzare tramite Suricata e lo script Lua e restituisce **o le suddette stringhe o un insieme di caratteri casuali dalla lunghezza che va da 5 a 10.**

```
def random_payload(payloads):  
    length = random.randint(5, 15)  
    random_string = ''.join(random.choices(string.ascii_lowercase, k=length))  
    if random.random() < 1:  
        random_payload = random.choice(payloads)  
        return random_payload  
    return random_string
```

Funzione genera-payload


```
payloads = [
    "CPU Usage: " + str(random.randint(40,100)) + "%",
    "Temperature: " + str(random.randint(40,100)) + "C",
    "Humidity: " + str(random.randint(40,100)) + "%",
    "Pressure: " + str(random.randint(40,1000)) + "hPa",
    "Noise level: " + str(random.randint(40,100)) + "dB",
    "Air Quality Index: " + str(random.randint(40,100))
]
```

Formato payload in Python

Prima di procedere è necessario abilitare il protocollo Modbus all'interno del file di configurazione [suricata.yaml](#), andando a specificare come porta da analizzare la 502.

Da sottolineare che nel file, la porta nominata poco fa è già impostata di default dato che la **porta server TCP standardizzata per Modbus-TCP** è la proprio la **502**.

```
modbus:
  # How many unanswered requests to flood
  # If the limit is reached, the request will be dropped
  #request-flood: 500

  enabled: yes
  detection-ports:
    dp: 502
```

Configurazione
suricata.yaml

In seguito verrà usata come macchina host Kali Linux insieme ad una **macchina Vagrant** con installato **debian/bullseye64**. Quest'ultima è formata da **client, R1, R2, server**. Sulla macchina R1 verranno installati Suricata e da lì sarà avviato il client Modbus che comunica con il server Modbus situato sull'host Kali e su entrambe le macchine dovrà essere installato Python insieme e pyModbus.

Per intercettare il traffico verrà utilizzata la stessa regola utilizzata per il rilevamento dei pacchetti in Scapy, cambiando da protocollo **tcp** a **modbus** e verrà chiamata [lua modbus.rules](#).

Per lanciare il server ed il client è necessario eseguire rispettivamente i seguenti comandi: **python3 [server_payloads.py](#) -c tcp -p 502** e **python3 [client_payloads.py](#) -c tcp -p 502 --host 192.168.1.61**.

- **c:** indica il protocollo da utilizzare per la comunicazione.
- **p:** porta su cui insiste il server.
- **host:** indirizzo IP dove si trova il servizio sul server.

Raccomando di avviare client e server direttamente dalla cartella examples della [repository](#) messa a disposizione da pyModbus.

Il server deve essere avviato prima del client, altrimenti quest'ultimo non potrà funzionare. Al momento dell'invio del payload, lo script lua analizzerà i valori inerenti ai dati sensibili, quali **temperatura, utilizzo della CPU e umidità**, e farà scattare la regola Suricata secondo le precedenti condizioni.

Controllando il file di log **eve.json** ci sarà la conferma che tutto si è svolto come previsto e l'alert generato fornirà una diagnosi dell'anomalia rilevata.

```
{
  "timestamp": "2023-08-25T03:08:13.649317+0000",
  "flow_id": 1717568613926821,
  "in_iface": "eth0",
  "event_type": "alert",
  "src_ip": "192.168.1.201",
  "src_port": 502,
  "dest_ip": "10.0.2.15",
  "dest_port": 54714,
  "proto": "TCP",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 101,
    "rev": 1,
    "signature": "Too much CPU usage/high temperature/high humidity. Possible attack!!!",
    "category": "",
    "severity": 3
  },
  "app_proto": "modbus",
  "flow": {
    "pkts_toserver": 1523,
    "pkts_toclient": 3041,
    "bytes_toserver": 160800,
    "bytes_toclient": 248838,
    "start": "2023-08-25T03:08:12.280485+0000"
  },
  "payload_printable": ".....\nCPU Usage: 70%\nHumidity: 95%\nTemperature: 64C\nPressure: 47hPa\nPressure: 100hPa\nPressure: 74hPa\nHumidity: 53%\nPressure: 50hPa\n",
  "stream": 0
}
```

Grazie alla funzione **payload_printable** abilitabile tramite il file di configurazione **suricata.yaml**, il log **eve.json** diventa un tool molto potente e supera in comodità fast.log, in quanto contiene anche il payload. Ciò permette di capire al meglio la natura del problema.

Conclusioni

Durante il corso di questo tirocinio con l'UlisseLab, ho avuto l'opportunità di lavorare con diversi strumenti e tecnologie, sviluppando una comprensione profonda di [Suricata](#), [Scapy](#), [Lua](#), [PyModbus](#) e il protocollo [Modbus](#).

Tra le molte cose che ho imparato, **Suricata** si è rivelata una risorsa preziosa per l'analisi del traffico di rete. Con esso, ho acquisito competenze essenziali nella rilevazione e logging del traffico di rete grazie alle Suricata Rules e Keywords. Inoltre, ho imparato a creare, testare e implementare regole efficaci per identificare e reagire alle attività sospette.

Con l'utilizzo di **Scapy**, ho appreso il potenziale di questa potente **libreria Python** per la manipolazione dei pacchetti di rete. Ho usato Scapy per analizzare, creare e inviare i miei propri pacchetti, offrendomi una comprensione pratica e approfondita delle interazioni di rete a basso livello.

Attraverso l'apprendimento del linguaggio di programmazione **Lua**, completamente nuovo per me ad inizio tirocinio, in combinazione con Suricata, sono riuscito a personalizzare e ottimizzare ulteriormente le funzionalità di Suricata. Questo mi ha permesso di gestire i dati di rete in modo più efficiente e di rispondere con maggiore flessibilità ai vari scenari di sicurezza della rete.

Ho affinato ulteriormente le mie competenze tecniche con l'uso di **pyModbus** che mi ha permesso di lavorare con il protocollo Modbus. Questa esperienza mi ha dato una comprensione approfondita delle comunicazioni industriali e mi ha permesso di interagire con dispositivi Modbus, leggendo e scrivendo nei registri.

Attraverso l'apprendimento e l'implementazione del protocollo Modbus, ho capito come vengono gestite le comunicazioni tra i dispositivi in un ambiente industriale. Ho appreso come i dati vengono inviati e ricevuti, come vengono gestiti gli errori, e ho compreso l'importanza dei registri Modbus e il loro utilizzo nel monitoraggio e controllo dei dispositivi.

Nel complesso, i principali obiettivi raggiunti durante questo tirocinio includono l'acquisizione di una comprensione approfondita del funzionamento del protocollo Modbus e dei principi della comunicazione industriale, lo sviluppo di competenze pratiche nell'uso di strumenti e linguaggi di programmazione come **Lua** e **Python**. Dopodiché è stato possibile implementare soluzioni personalizzate per l'analisi del traffico di rete e la rilevazione delle intrusioni (tramite l'**interazione tra Suricata e Lua**) e applicare le competenze acquisite nel mondo reale (grazie a pyModbus), migliorando l'efficienza e la sicurezza della rete.

In conclusione, questo tirocinio è stato una preziosa opportunità per acquisire competenze pratiche e conoscenze teoriche in ambito di sicurezza delle reti e protocolli di comunicazione industriale. Queste competenze e conoscenze saranno senza dubbio di grande utilità nella mia futura carriera nell'ambito della sicurezza informatica e della sicurezza industriale.